In RAG We Trust? Ensamble Models for Retrieval Augmented Generation

Christian R. Cuenca
Dipartimento di Informatica
Università degli Studi di Milano
Milan, Italy
christian.cuenca@studenti.unimi.it

***Abstract*—Lorem ipsum**
***Index Terms*—RAG, LLM Ensembling, Poison Issue**

I. Introduction

II. Materials and Methods

A. Dataset

The dataset used in this study is synthetic, i.e., it was artificially generated using a commercial LLM, specifically OpenAI's ChatGPT 5.2. The reference corpus consists of 10 documents. For ease of handling within the preprocessing pipeline, all documents are stored in plain-text format (`.txt`). In a real-world deployment, the same material would typically be distributed as PDFs or other heterogeneous formats, requiring more advanced preprocessing (e.g., layout parsing and robust text extraction).

The documents are designed to reflect enterprise management documentation in a corporate setting, with a focus on the banking domain. Concretely, they simulate internal policies and operational procedures relevant to an ICT organizational unit, including security and governance oriented guidance. In operational contexts, such documents play a central role in organizational decision-making, particularly for information security and compliance-related workflows.

In terms of size, documents contain on average 984.3 words. The longest document includes $1,865$ words, while the shortest includes 823 words. Structurally, each document is organized into numbered sections with headings, and some documents include additional subsections to mirror common corporate documentation conventions.

B. Document preprocessing

The dataset is processed by a `Chunker` module whose objective is to segment each `.txt` document into section-level chunks based on numbered headings (e.g., "1. Introduction"). To improve interpretability and facilitate manual inspection, each extracted section is also exported as an individual `.txt` file while preserving associated metadata. Specifically, each chunk contains all lines belonging to a given section, including the section number and title, and is stored without semantic rewriting, normalization, nor content cleaning.

The original ordering of sections is retained through a progressive indexing scheme (e.g., `chunk_001`, `chunk_002`). Each chunk is additionally linked to its source document via metadata that records the original filename, enabling traceability from retrieved passages back to the originating document.

The `Collector` module reads each generated chunk, extracting both its semantic content and the associated metadata, and then computes dense sentence-level embeddings using the lightweight Sentence-Transformers model `all-MiniLM-L6-v2`. The resulting vectors, together with the corresponding metadata, are stored in a persistent `ChromaDB` collection, enabling similarity-based retrieval.

Embeddings are computed only from the section body text, excluding the section number and title. This design choice aims to improve the semantic quality of the vector representations, and enhance query generalization during retrieval.

C. Inference flow

At inference time, the user provides a natural-language question to the system. The question is first handled by the `Query Augmentor`, an LLM-based module that reformulates the original input into one or more enhanced retrieval queries. The goal is to improve retrieval effectiveness by emphasizing semantically informative terms and reducing the impact of vague or low-signal tokens that could otherwise degrade similarity matching.

The `Retriever` then encodes the enhanced queries into dense vectors using the same embedding model adopted during indexing and performs semantic vector search over the ChromaDB collection. Similarity is computed using cosine similarity, and nearest-neighbor retrieval is implemented via HNSW (Hierarchical Navigable Small World), which enables efficient approximate search in high-dimensional embedding spaces. The retriever returns the top-k most similar vectors along with their associated metadata, which are subsequently used to construct the evidence context for downstream generation.

The original user question, together with the retrieved context from the vector database, is provided to a Council of `Augmentor` models. Each Augmentor is an LLM instantiated with a role-specific system prompt designed to elicit complementary perspectives over the same evidence. For instance, prompts include roles such as *a conservative banking compliance expert, a pragmatic ICT operations engineer, and a critical security auditor*.

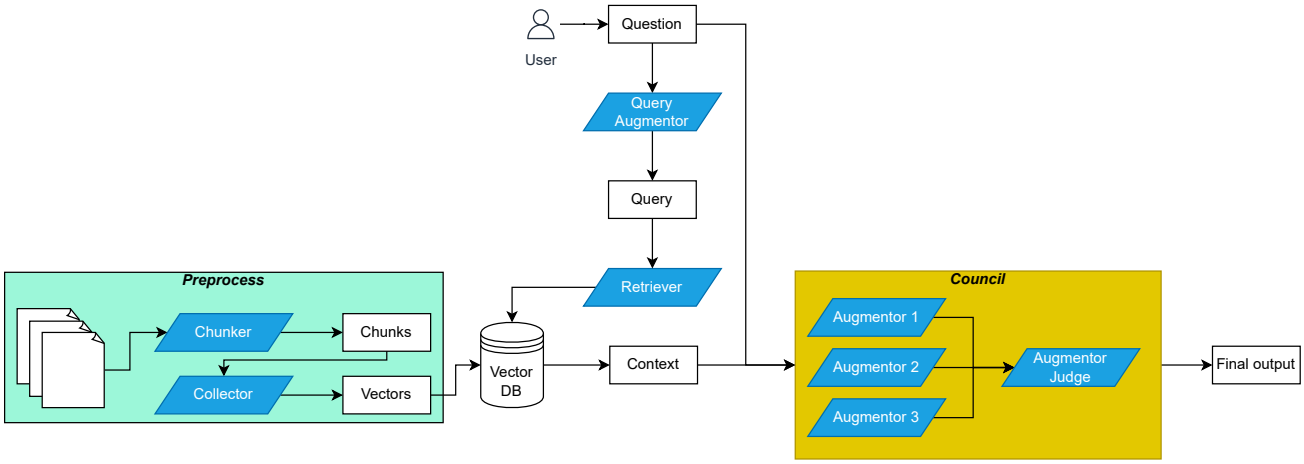

Fig. 1: Overview of the proposed RAG-based pipeline. During preprocessing, documents are segmented into chunks by the Chunker. The Collector encodes each chunk into a dense vector and indexes it in the vector database. At inference time, a user question is first reformulated by a query-enhancement model, which generates one or more retrieval queries. The Retriever performs top-k similarity search over the vector index and aggregates the retrieved passages into a supporting context. The original question and the retrieved context are then provided to the Council, i.e., an ensemble of role-specialized generative models producing candidate answers. Finally, a Judge model validates these candidates and outputs the final response.

Augmentation is implemented through a structured prompt template shared across all Augmentor models as follow:

> You are {*role prompt*}
> Answer using ONLY the context below.
> Context: {*context*}
> Question: {*question*}

This formulation enforces the evidence base by discouraging the use of outside knowledge and ensures that differences in candidate responses arise primarily from the specific definition of the role rather than from access to different sources.

Finally, all candidate answers produced within the Council are reviewed by a dedicated `Augmentor Judge`, whose objective is to consolidate the Council's outputs into a single response. The Judge analyzes (i) the original user question, (ii) the retrieved evidence context, and (iii) the set of candidate answers, with the explicit goal of resolving inconsistencies and selecting the most defensible interpretation supported by the evidence. The Judge is prompted as follows:

> You are a senior banking ICT and security reviewer.
> Question: {*question*}
> Context: {*context*}
> Below are answers from multiple experts:
> {*multi-answers*}
> Task:
> - Compare the answers
> - Resolve contradictions
> - Produce a single, accurate, well-justified final answer
> - Use ONLY the provided context

After this step, the court's consolidated output is returned as the system's final answer.

III. Results

IV. Discussion and conclusion

Acknowledgment

References